

University of Asia Pacific

Department of CSE

Assignment On

SJF Preemptive (CPU Scheduling Algorithms)

Course Title: Operating System Lab

Course Code: CSE 402

Submitted To

Atia Rahman Orthi

Lecturer

Department of Computer Science

and Engineering

University of Asia Pacific

Submitted By

Sultan Ruhul Kuddus Nayan

ID: 23101159

2022–2023 Batch

Department of CSE

University of Asia Pacific

Code Implementation

```
processes = [  
    ["P1", 4, 2],  
    ["P2", 2, 2],  
    ["P3", 1, 3],  
    ["P4", 0, 6],  
    ["P5", 3, 1]  
]  
  
quantum = 2  
n = len(processes)  
  
remaining = {}  
  
for p in processes:  
    remaining[p[0]] = p[2]  
  
CT = {}  
sequence = []  
time = 0  
  
while len(CT) < n:  
  
    available = []  
  
    for p in processes:  
        name = p[0]  
        AT = p[1]  
  
        if AT <= time and remaining[name] > 0:  
            available.append(p)  
  
    if len(available) == 0:  
        time += 1  
        continue  
  
    current = min(  
        available,  
        key=lambda p: remaining[p[0]]  
    )  
  
    name = current[0]  
  
    sequence.append(name)  
  
    run_time = min(quantum, remaining[name])  
  
    remaining[name] -= run_time
```

```

        time += run_time

        if remaining[name] == 0:
            CT[name] = time

TAT = {}
WT = {}

for p in processes:

    name = p[0]
    AT = p[1]
    BT = p[2]

    TAT[name] = CT[name] - AT
    WT[name] = TAT[name] - BT


print("Process Execution Sequence:")
print(" -> ".join(sequence))


print("\n")
print(f"{'P':<8}{'AT':<8}{'BT':<8}{'CT':<8}{'TAT':<8}{'WT':<8}")
print("-" * 48)

total_tat = 0
total_wt = 0

for p in processes:

    name = p[0]
    AT = p[1]
    BT = p[2]

    print(
        f"{name:<8}"
        f"{AT:<8}"
        f"{BT:<8}"
        f"{CT[name]:<8}"
        f"{TAT[name]:<8}"
        f"{WT[name]:<8}"
    )

    total_tat += TAT[name]
    total_wt += WT[name]

```

```
avg_tat = total_tat / n
avg_wt = total_wt / n

print("-" * 48)
print(f"Average TAT = {avg_tat:.2f} sec")
print(f"Average WT  = {avg_wt:.2f} sec")
```

Discussion

The program successfully implements a CPU scheduling technique using a **time quantum of 2 seconds**. The processes are selected based on their remaining burst time among the available processes. Each selected process is allowed to execute for a maximum of 2 seconds. If the process is not completed, its remaining burst time is updated and it is considered again for execution.

For the given processes, the execution sequence is **P4 → P2 → P5 → P1 → P3 → P3 → P4 → P4**. The program calculates Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process. The calculated **average Turnaround Time is 6.00 seconds**, and the **average Waiting Time is 3.20 seconds**.

The result shows how CPU scheduling can manage multiple processes efficiently by dividing CPU execution into smaller time intervals. However, it should be noted that the implementation is not a pure Round Robin algorithm because it selects the process with the shortest remaining burst time instead of strictly following a FIFO ready queue.

Conclusion

In conclusion, this assignment provides a practical understanding of CPU scheduling and process management using Python. The program successfully calculates CT, TAT, and WT and displays the process execution sequence. The obtained average TAT of **6.00 seconds** and average WT of **3.20 seconds** demonstrate the scheduling performance for the given processes.

The implementation also helps to understand the importance of **Arrival Time, Burst Time, Time Quantum, Completion Time, Turnaround Time, and Waiting Time** in CPU scheduling. Overall, the assignment improves the understanding of preemptive scheduling concepts and provides a foundation for implementing standard Round Robin and other CPU scheduling algorithms.

Git Repository : <https://github.com/sultan36912/Operating-System-Lab..git>